

Unit B4: Abstract Data Types (HL Only)

Sub-topics: B4.1 Fundamentals of ADTs

1. Syllabus Scope & Learning Outcomes

This unit is exclusively for Higher Level students. It focuses heavily on how data is structured in memory.

Syllabus Point	Command Term	Student Expectation (The "Do")
B4.1.1	Explain	Describe the purpose of ADTs: separating the <i>interface</i> (what it does) from the <i>implementation</i> (how it works).
B4.1.2	Evaluate	Compare Linked Lists vs. Arrays regarding memory efficiency and access speed.
B4.1.3	Construct / Apply	Sketch diagrams and write Python code for Singly, Doubly, and Circular Linked Lists (insert, delete, traversal).
B4.1.4	Explain	Describe BST structure and trace traversals (In-order, Pre-order, Post-order).
B4.1.5	Construct / Apply	Use Set theory and Python Sets to perform Union, Intersection, and Difference operations.
B4.1.6	Explain	Explain Hash Table mechanics: Hashing algorithms, Collisions (Chaining/Open

		Addressing), and Load Factors.
--	--	--------------------------------

2. Key Terminology & Definitions

- **ADT (Abstract Data Type):** A conceptual model where the data type is defined by its behaviour (operations) rather than its implementation.
- **Pointer/Reference:** A variable that stores the memory address of another variable/node.
- **Node:** The basic unit of a data structure (e.g., Linked List or Tree), containing data and pointer(s).
- **Head / Tail:** The first and last nodes of a Linked List.
- **Root / Leaf:** The top node and the bottom nodes (no children) of a Tree.
- **Traversal:** Visiting every node in a data structure exactly once.
- **Recursion:** (Review from B2) Essential for traversing Trees.
- **Hashing Algorithm:** A function that converts input (Key) into a fixed-size integer (Index).
- **Collision:** When two different keys hash to the same index.
- **Load Factor:** The ratio of stored elements to available slots in a Hash Table (triggers rehashing if too high).

3. Core Content & Exam Actions

B4.1.1: Purpose of ADTs

Theory:

- **Abstraction:** The user knows *what* pop() does, but doesn't need to know *how* the pointer logic works.
- **Encapsulation:** Data integrity is maintained; external code cannot corrupt the internal structure directly.

Student Exam Action:

- **Explanation:** Explain why using an ADT improves code maintenance (implementation can change without breaking the rest of the program).

B4.1.2 & B4.1.3: Linked Lists

Theory:

- **Structure:** A chain of Nodes. Each Node = [Data | Pointer].
- **Types:**
 - **Singly:** Points forward only (Next).
 - **Doubly:** Points forward (Next) and backward (Prev).
 - **Circular:** The Tail points back to the Head.
- **vs Arrays:**
 - *Arrays:* Static size, fast access (index), slow insertion (shifting).

- *Linked Lists*: Dynamic size, slow access (sequential), fast insertion (pointer swap).

Student Exam Action:

- **Diagramming**: Draw a Linked List before and after inserting a node "X" between nodes "A" and "B". (Show the pointer arrows changing).
- **Coding**: Write a Python Node class and a LinkedList class with an insert_at_head(data) method.

B4.1.4: Binary Search Trees (BST)

Theory:

- **Rules**: Left Child < Parent < Right Child.
- **Traversals**:
 - **In-order (LNR)**: Left, Node, Right (Outputs sorted data: A-Z).
 - **Pre-order (NLR)**: Node, Left, Right (Useful for copying trees).
 - **Post-order (LRN)**: Left, Right, Node (Useful for deleting trees).

Student Exam Action:

- **Sketching**: Given a list of numbers [10, 5, 15, 3, 7], draw the resulting BST.
- **Tracing**: Write the output sequence for a Post-order traversal of a given tree diagram.

B4.1.5: Sets

Theory:

- **Properties**: Unordered, Unique elements (no duplicates), Mutable.
- **Operations**:
 - **Union** ($A \cup B$): All elements from both.
 - **Intersection** ($A \cap B$): Elements present in BOTH.
 - **Difference** ($A - B$): Elements in A that are NOT in B.
 - **Subset** ($A \subseteq B$): If all elements of A are in B.

Student Exam Action:

- **Calculation**: Given Set $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, determine $A \cup B$ and $A \cap B$.
- **Coding**: Use Python methods .union(), .intersection(), and checks like if x in my_set:.

B4.1.6: Hash Tables

Theory:

- **Mechanism**: $Index = Key \pmod{Size}$.
- **Collision Resolution**:

- **Chaining:** The index holds a Linked List of all items hashing to that slot.
- **Open Addressing:** If the slot is full, look for the next empty slot (Linear Probing).
- **Rehashing:** If the Load Factor > 0.7 (70% full), create a larger table and re-insert all items to reduce collisions.

Student Exam Action:

- **Calculation:** Calculate the hash index for a given Key using a Modulo formula.
- **Diagramming:** Draw a Hash Table using Chaining after inserting a specific sequence of keys, showing where collisions occur.

4. Common Pitfalls & Examiner Tips

- **Pitfall (Linked List Insertion):** Students often break the chain. When inserting Node Z between A and B, you must point Z to B *before* pointing A to Z. If you point A to Z first, you lose the link to B (and the rest of the list).
- **Pitfall (BST Insertion):** Do not re-balance the tree unless asked. If the input is 1, 2, 3, 4, the BST looks like a straight line (linked list), which is inefficient but correct for a basic BST.
- **Pitfall (Hashing):** Confusing the *Key* (input data) with the *Index* (storage location).
- **Tip (Traversals):** Memorise the acronyms (LNR, NLR, LRN). "In" means Node is **In**-between. "Pre" means Node is **Pre**-vious (first). "Post" means Node is **Post**-erior (last).